

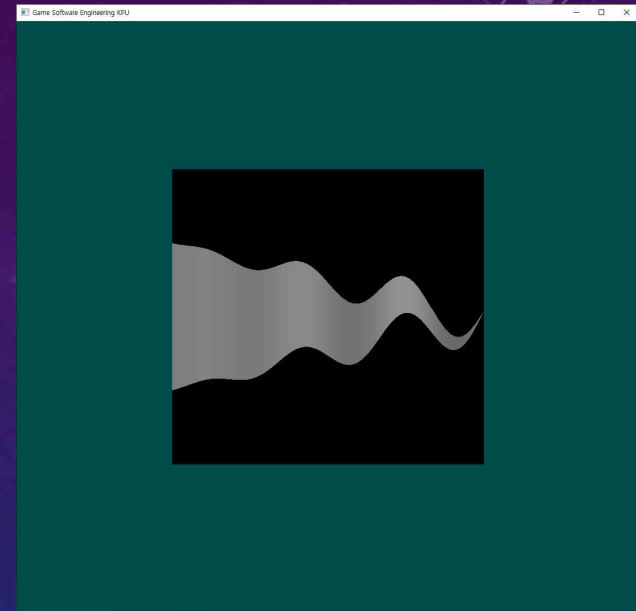


셰이더프로그래밍

LECTURE 6

이택희

버텍스 셰이더 사용 애니메이션

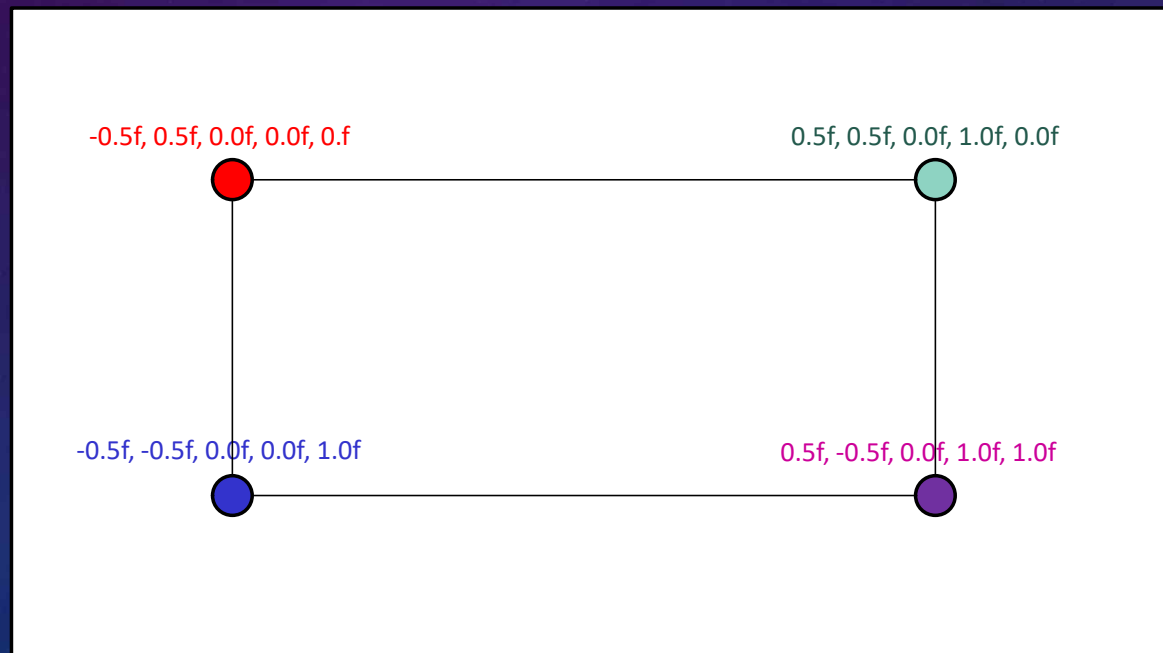


개요

- 버텍스 셰이더 사용 애니메이션
 - 수많은 삼각형들로 이루어진 메시를 사용
 - 버텍스 셰이더를 이용하여 효과 구현
 - 물결
 - 깃발

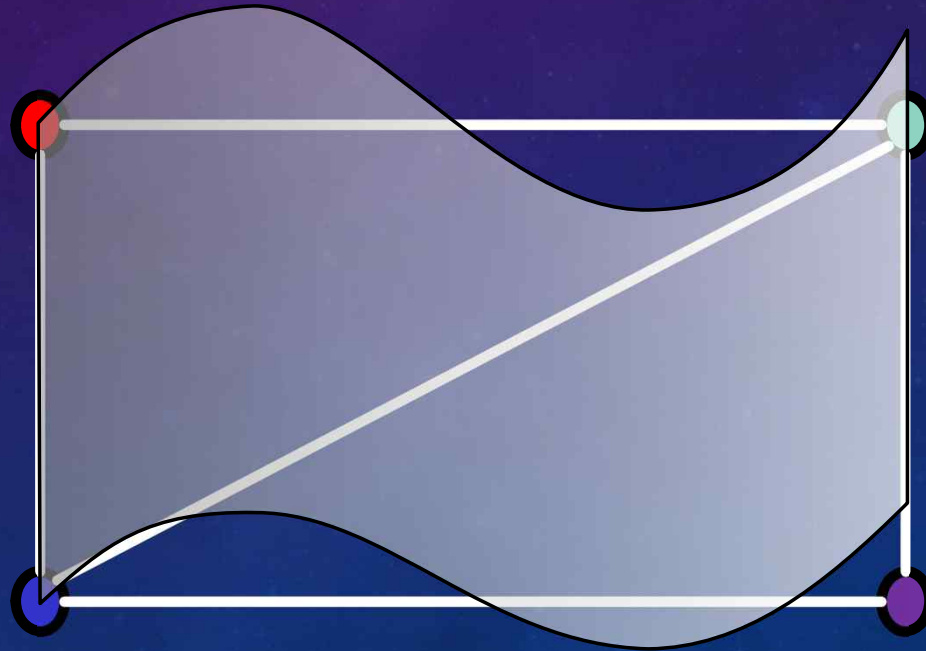
버텍스 셰이더 응용

- 지금까지는 적은 수의 버텍스 들만 사용
 - 주로 Fragment shader 실습을 위한 내용



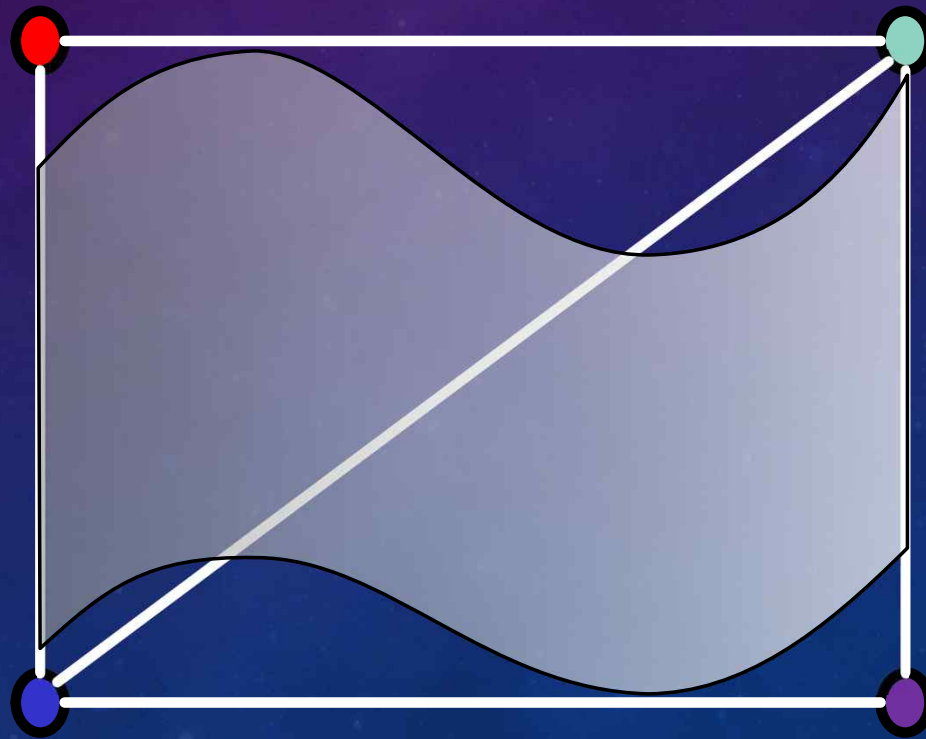
버텍스 셰이더 응용

- 적은 수의 버텍스 만으로 아래와 같이 물결 치는 표현이 가능할까?



버텍스 셰이더 응용

- 버텍스가 커버하는 프래그먼트 영역이 충분히 크다면 프래그먼트 셰이더로 가능

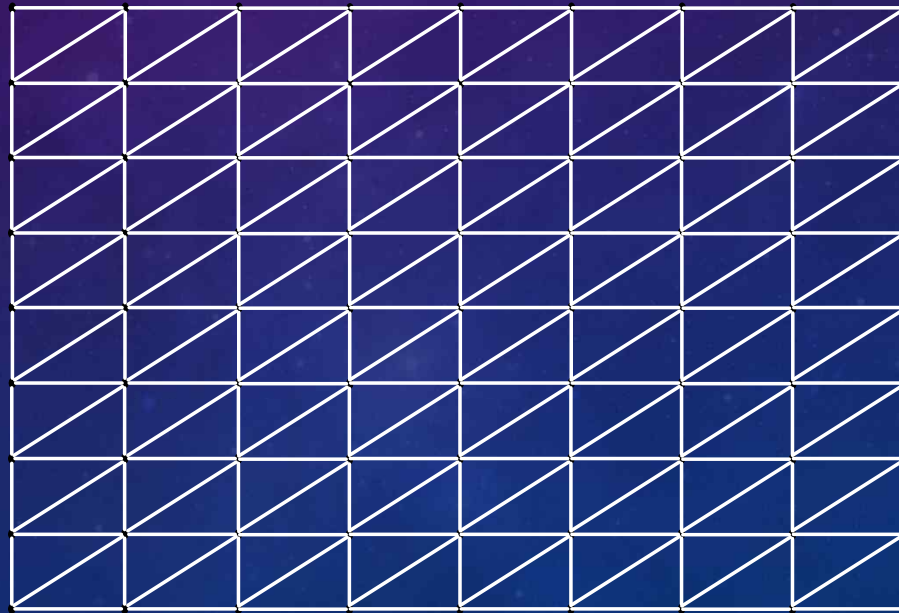


버텍스 셰이더 응용

- 프래그먼트 셰이더 기반
 - 프래그먼트 취소작업 필요
 - 많은 계산 량
 - 화면에 500x500 프래그먼트 커버 시 500x500만큼의 연산 필요

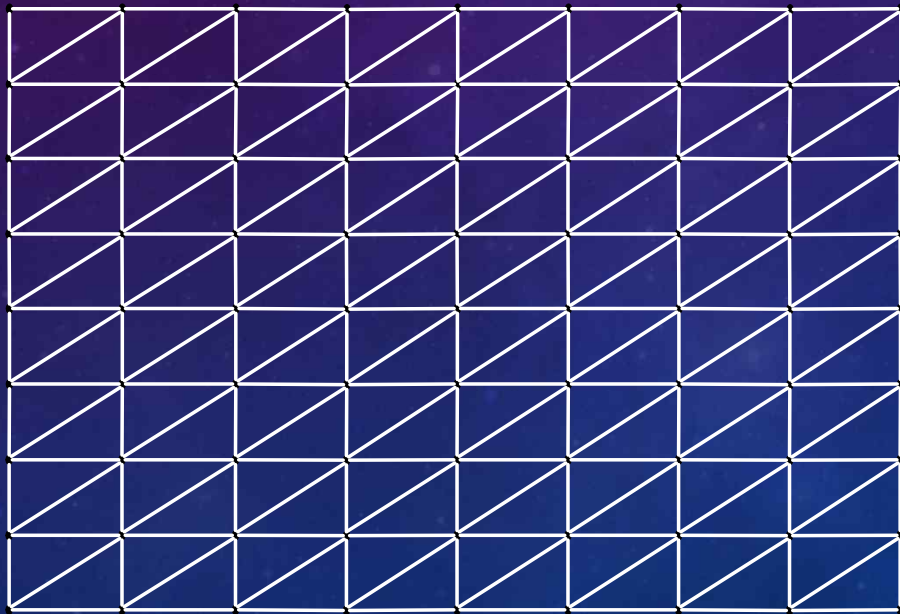
버텍스 셰이더 응용

- 깃발이 펄럭이는 효과를 버텍스 셰이더에서 구현



버텍스 셰이더 응용

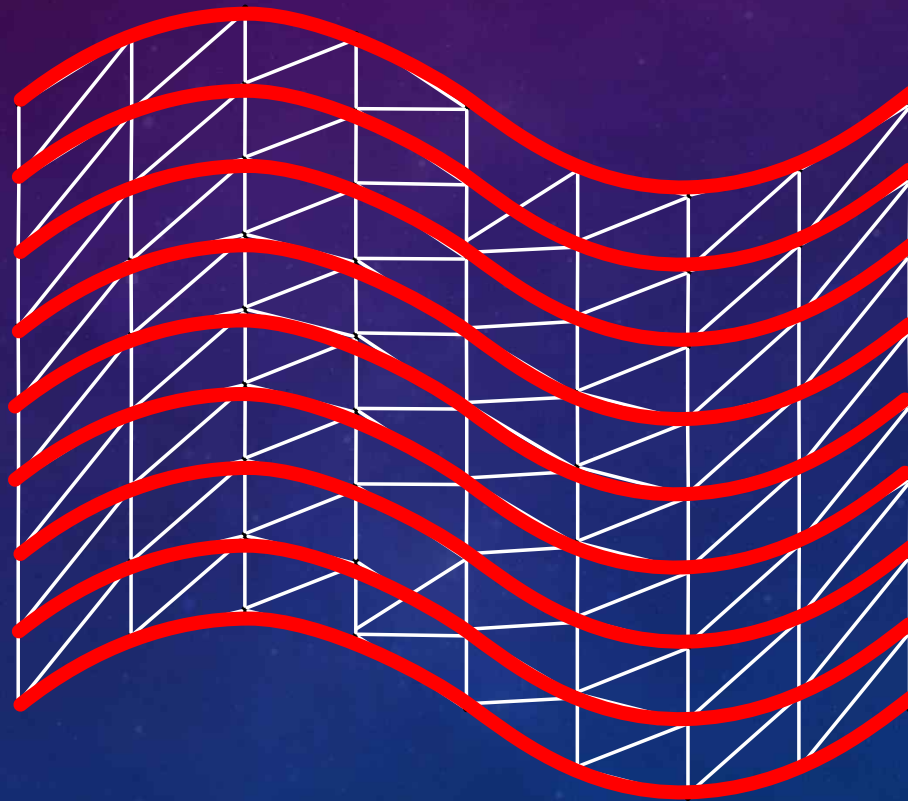
- 깃발이 펄럭이는 효과를 버텍스 셰이더에서 구현



8X8 개의 사각형
8X8X2 개의 삼각형
8X8X2X3 개의 버텍스

즉, 8X8X2X3 번의 버텍스
연산이 필요

버텍스 셰이더 응용



버텍스 셰이더 응용(실습 준비)

Vertex Shader

```
#version 330

in vec3 Position;

void main()
{
    gl_Position = vec4(Position, 1.0);
}
```

Fragment Shader

```
#version 330

out vec4 FragColor;

void main()
{
    FragColor = vec4(1.0, 1.0, 1.0, 1.0);
}
```

버텍스셰이더 응용(실습 준비)

```

float basePosX = 0.5f;
float basePosY = 0.5f;
float targetPosX = 0.5f;
float targetPosY = 0.5f;

int pointCountX = 8;
int pointCountY = 8;

float width = targetPosX - basePosX;
float height = targetPosY - basePosY;

float* point = new float[pointCountX*pointCountY*2];
float* vertices = new float[(pointCountX - 1)*(pointCountY - 1)* 2 * 3];
gDummyVertexCount = (pointCountX - 1)*(pointCountY - 1) * 2 * 3;

//Prepare points
for (int x = 0; x < pointCountX; x++)
{
    for (int y = 0; y < pointCountY; y++)
    {
        point[(y*pointCountX+x) * 2 + 0] = basePosX + width * (x / (float)(pointCountX - 1));
        point[(y*pointCountX+x) * 2 + 1] = basePosY + height * (y / (float)(pointCountY - 1));
    }
}

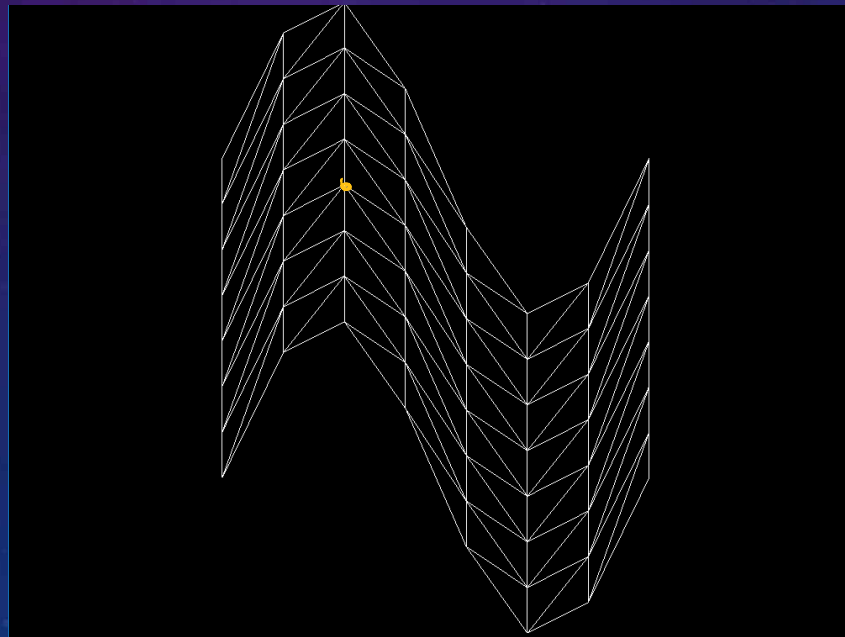
//Make triangles
int vertindex = 0;
for (int x = 0; x < pointCountX - 1; x++)
{
    for (int y = 0; y < pointCountY - 1; y++)
    {
        //Triangle part 1
        vertices[vertindex] = point[(y*pointCountX+x) * 2 + 0];
        vertindex++;
        vertices[vertindex] = point[(y*pointCountX+x) * 2 + 1];
        vertindex++;
        vertices[vertindex] = 0.f;
        vertindex++;
        vertices[vertindex] = point[((y + 1)*pointCountX + (x + 1)) * 2 + 0];
        vertindex++;
        vertices[vertindex] = point[((y + 1)*pointCountX + (x + 1)) * 2 + 1];
        vertindex++;
        vertices[vertindex] = 0.f;
        vertindex++;
        vertices[vertindex] = point[((y + 1)*pointCountX + x) * 2 + 0];
        vertindex++;
        vertices[vertindex] = point[((y + 1)*pointCountX + x) * 2 + 1];
        vertindex++;
        vertices[vertindex] = 0.f;
        vertindex++;
        vertices[vertindex] = point[((y + 1)*pointCountX + (x + 1)) * 2 + 0];
        vertindex++;
        vertices[vertindex] = point[((y + 1)*pointCountX + (x + 1)) * 2 + 1];
        vertindex++;
        vertices[vertindex] = 0.f;
        vertindex++;
        vertices[vertindex] = point[(y*pointCountX+x) * 2 + 0];
        vertindex++;
        vertices[vertindex] = point[(y*pointCountX+x) * 2 + 1];
        vertindex++;
        vertices[vertindex] = 0.f;
        vertindex++;
        vertices[vertindex] = point[(y*pointCountX+x) * 2 + 0];
        vertindex++;
        vertices[vertindex] = point[(y*pointCountX+x) * 2 + 1];
        vertindex++;
        vertices[vertindex] = 0.f;
        vertindex++;
        vertices[vertindex] = point[(y*pointCountX+x) * 2 + 0];
        vertindex++;
        vertices[vertindex] = point[(y*pointCountX+x) * 2 + 1];
        vertindex++;
        vertices[vertindex] = 0.f;
        vertindex++;
    }
}

glGenBuffers(1, &VBO_DummyMesh);

```

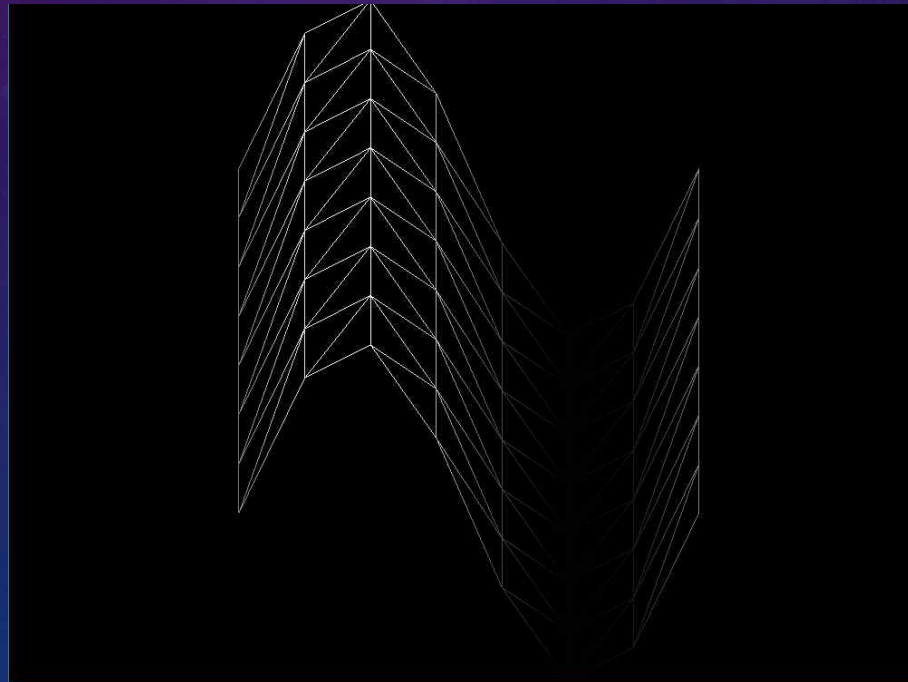
실습

- 내용 구현
- 버텍스 셰이더에서 $\sin(\text{radian})$ 함수를 사용하여 아래와 같은 그림 출력



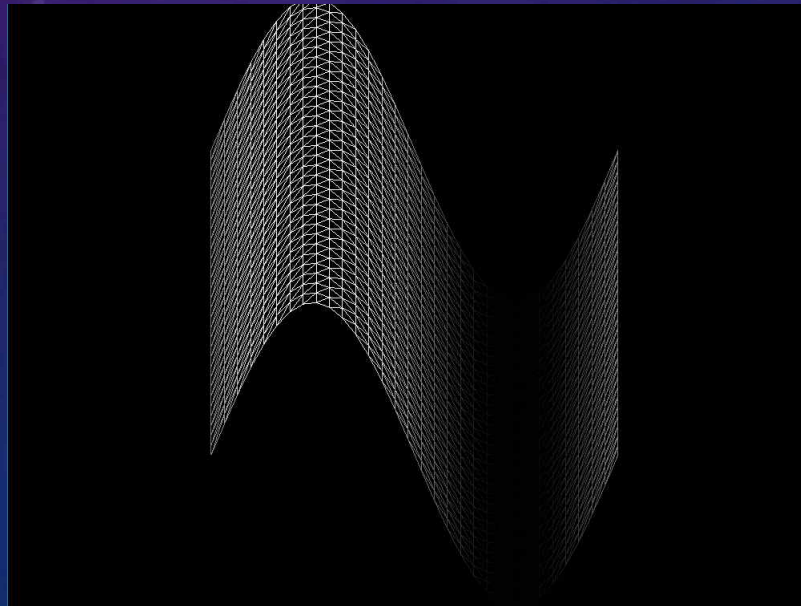
실습

- 버텍스 셰이더 out 및 프래그먼트 셰이더 in 을 적용하여 아래와 같이 컬러 적용



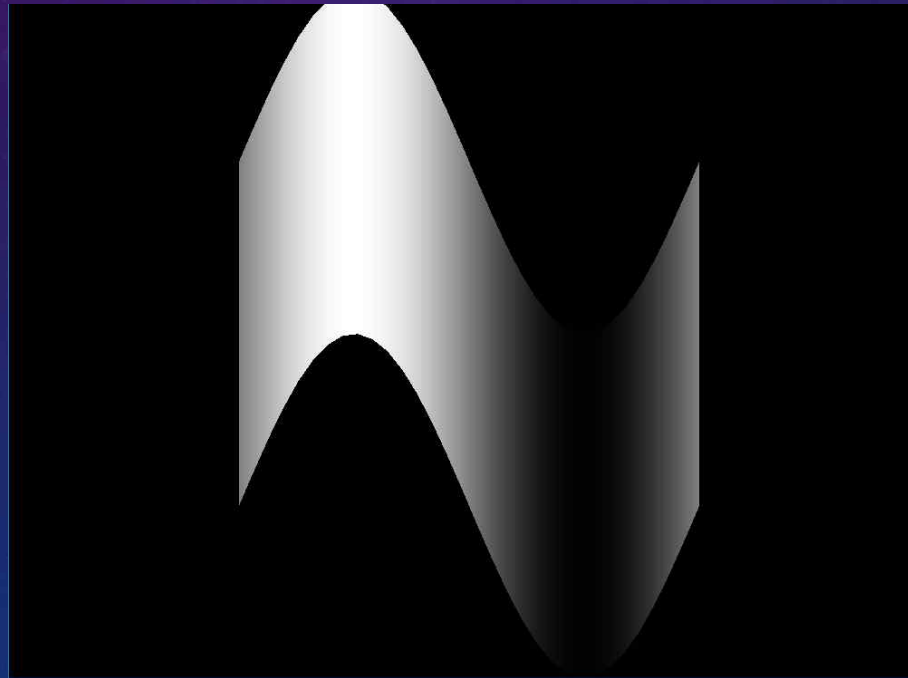
실습

- 메시를 좀더 잘게 쪼개어 보자
 - `int pointCountX = 32;`
 - `int pointCountY = 32;`



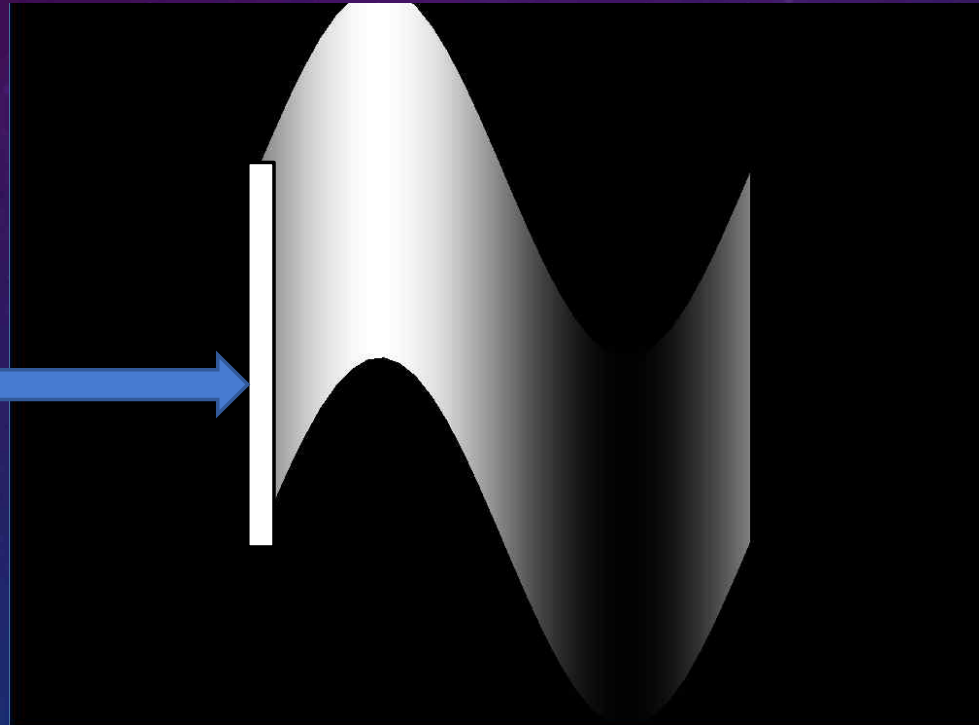
실습

- Primitive 를 아래와 같이 변경
 - GL_TRIANGLES



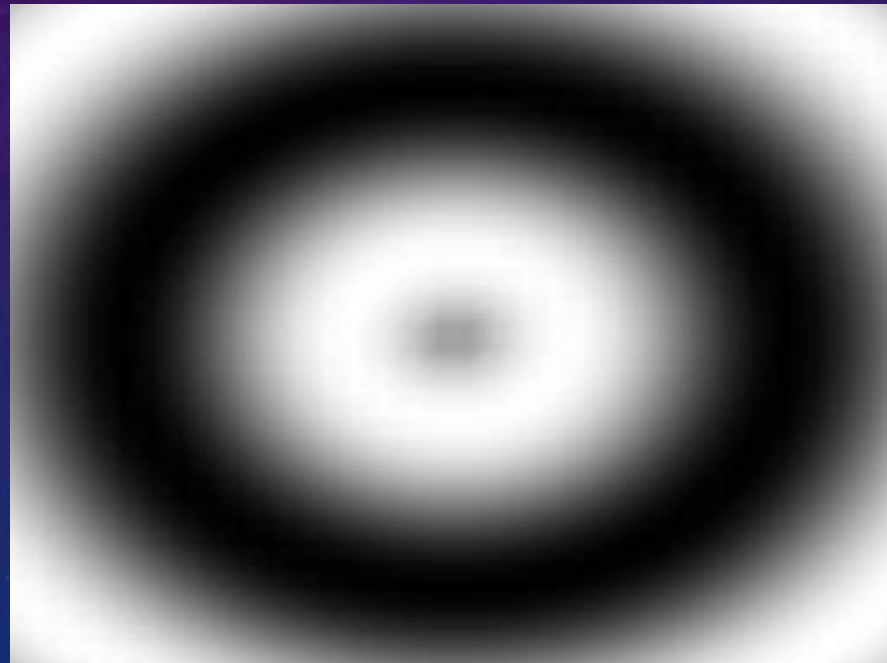
실습

고정



실습

- 화면 가운데를 중심으로 물결이 퍼져 나가는 효과 구현



실습

$$\begin{aligned}x &= r \cos \theta = \rho \sin \Phi \cos \theta \\y &= r \sin \theta = \rho \sin \Phi \sin \theta \\z &= \rho \cos \Phi\end{aligned}$$

